CLOUD COMPUTING MINI PROJECT

NAME :- NIRAJ KUMAR
Roll No :- 23010137
Github Repo : https://github.com/Niraj52118/cloud-assignment

1. Learning Journey and Project Understanding

Through this project, I developed an end-to-end understanding of how a machine-learning model can be prepared, packaged, deployed, monitored and evaluated as a cloud-native service. The application accepts natural language voice assistant commands and convert each command into a strict four-field Json response : intent, date, time, and place. This helped me connect concepts from machine learning with practical cloud-computing topics such as object storage, containers, orchestration, networking, availability, and automation.

Architectured learned

Phi-3.5-mini-instruct (3.8B parameters), served locally through Ollama with deterministic generation settings. acted as the teacher. EleutherAI Pythia-70M (70M parameter) was trained as the student through sequence-level knowledge distillation. SeaweedFs supplied S3-compatible storage for datasets and model artifacts. The FastAPI service was deployed in a local multi-node K3d Kubernetes cluster using a service.

Key Conceptual and Practical Learnings

- **Data pipeline and object storage:** I learned to divide teacher-labeling work into independent shards, store dataset and model files outside the container image and validate generated records before training. Cleaning the 3,000-command pool involve rejecting invalid JSON, malformed keys, unsupported intents, hallucinated slot values, and duplicates. The final verified dataset contained 2435 records, with a SHA-256 checksum for reproducibility.

- **Sequence-level Knowledge distillation**

I learned why directly matching teacher and student token probabilities was unsuitable for these different model architectures and tokenizers. instead, the teacher's generated JSON sequence were used as target examples for the student. This demonstrated how a compact model can learn output structure from a larger model.

- **Containerization and resource optimization**

I learned the importance of building a CPU-focused image instead of including unnecessary GPU libraries. Multi-stage Container construction and CPU-only Pytorch dependencies kept the image below the 2 GB

requirement, making it more practical to deploy on limited local hardware.

- **Kubernetes reliability and networking**

I gained practical experience with namespace, Deployment, services, ingress, readiness and liveness probes, rolling updates, rollbacks, pod recovery and horizontal scaling. The NACL task clarified that stateless rules must explicitly permit return traffic and must be ordered carefully.

- **CI/CD and reproducibility**

I learned how automated workflows can run codes checks, unit tests, evaluation, quality gat and secret scanning, reducing dependence on manual verification.